

Customer Churn Prediction Using Gradient Boosting

A Machine Learning Mini-Project

Submitted by: Mafeyisopin Ayeni

Course: Machine Learning | Assignment: Gradient Boosting | Date: 2026

Abstract

Churn is a big problem in the telecom industry. When customers leave, the company loses not just their payments but also the money spent to bring them in. In this project, I built a machine learning pipeline to predict which customers are likely to churn using the Telco Customer Churn dataset, which has 7,043 records and 21 features. I trained and compared two gradient boosting models: a standard GBM and a tuned XGBoost. Because the dataset was imbalanced (only 26% churners), I used `scale_pos_weight=3` in XGBoost to help it focus more on catching churners. XGBoost ended up with a Recall of 0.818 and ROC-AUC of 0.838, which made it the better model for this task. I also used SHAP to understand what was driving predictions and found that contract type, tenure and monthly charges were the main factors. Based on this, I came up with three business recommendations the company could act on.

Dataset link

<https://www.kaggle.com/datasets/blastchar/telco-customer-churn>

GitHub Repo

<https://github.com/mafeyisopin629-spec/customer-churn-prediction.git>

Self-declaration of originality

I, Mafeyisopin Ayeni, declare that this report and all accompanying code is my own original work. All sources used have been referenced accordingly. Date: 2026.

1. Part C: Practical Coding Exercises

C1: Titanic Survival Prediction

For this task I used the Titanic dataset from seaborn. After cleaning it and keeping only the useful columns (pclass, sex, age, sibsp, parch, fare, embarked), I was left with 714 rows. I used LabelEncoder to convert the text columns into numbers and then split the data 80/20 for training and testing. I trained both a Gradient Boosting model and Logistic Regression to compare them.

Metric	Gradient Boosting	Logistic Regression
Accuracy	0.769	0.797
Precision	0.778	0.854
Recall	0.667	0.651
F1 Score	0.718	0.739

Logistic Regression scored slightly higher on Accuracy and Precision, but Gradient Boosting had better Recall, meaning it was better at catching actual survivors. Logistic Regression works well here because the survival patterns are somewhat straightforward, but GBM handles the more complex interactions between features like age and passenger class better.

C2: Random Forest vs Gradient Boosting

I used the Breast Cancer dataset from sklearn (569 records, 30 features) to compare Random Forest and Gradient Boosting. I measured Accuracy, ROC-AUC and how long each one took to train.

Model	Accuracy	ROC-AUC	Train Time (s)
Random Forest	0.965	0.995	0.243
Gradient Boosting	0.956	0.995	0.769

Both models got the same ROC-AUC score of 0.995, so they were equally good at separating the two classes. The main difference was speed. Random Forest finished in 0.243 seconds while GBM took 0.769 seconds. This makes sense because Random Forest builds all its trees at the same time, while GBM builds them one after another where each tree tries to fix what the previous one got wrong.

C3: XGBoost with Early Stopping

Here I trained XGBoost on the Breast Cancer dataset and used two hyperparameters: `learning_rate=0.05` (so the model learns slowly and carefully) and `max_depth=4` (to keep the trees simple and avoid overfitting). I set `early_stopping_rounds=20` so training would stop if the validation loss did not improve for 20 rounds in a row.

The model stopped at iteration 109 with a best validation log loss of 0.1094. Looking at the training vs validation loss chart, the train loss kept going down but the validation loss levelled off around iteration 109, which is exactly when training stopped. This showed that early stopping was working properly and the model did not overfit.

2. Dataset, EDA and Methodology

2.1 About the Dataset

The Telco Customer Churn dataset from Kaggle has 7,043 rows and 21 columns. It covers customer demographics (gender, age, whether they have a partner or dependents), the services they are subscribed to (phone, internet, streaming, security, backup, tech support), their account details (how long they have been a customer, their contract type, payment method, monthly charges and total charges) and whether they churned or not.

When I loaded the data, the TotalCharges column had some blank spaces that were causing it to be read as text instead of numbers. I fixed this using `pd.to_numeric()` and dropped the 11 rows that had missing values, leaving 7,032 rows for the analysis.

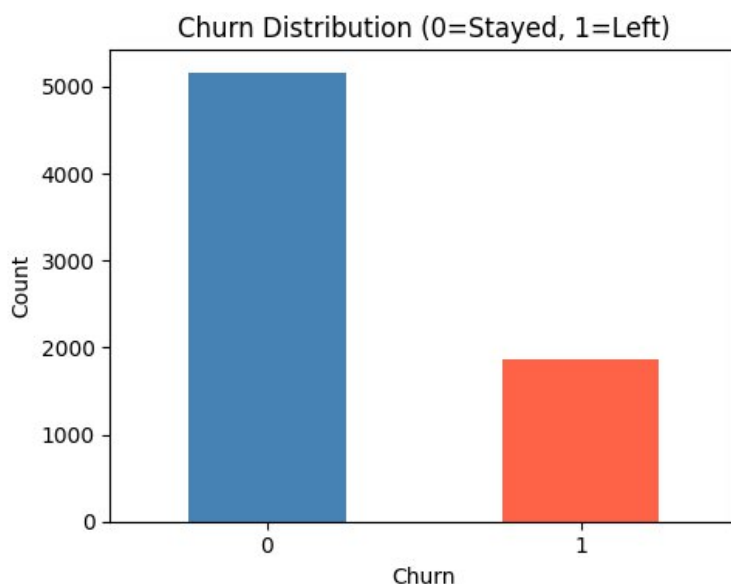
2.2 Class Imbalance

Out of 7,032 customers, 5,163 did not churn (73.5%) and 1,869 did churn (26.5%). This imbalance matters because a model could just predict nobody churns and still get 74% accuracy, which would be useless in practice. To deal with this, I set `scale_pos_weight=3` in XGBoost (calculated from 5163 divided by 1869, which rounds to about 3). This tells the model to treat churners as 3 times more important when learning.

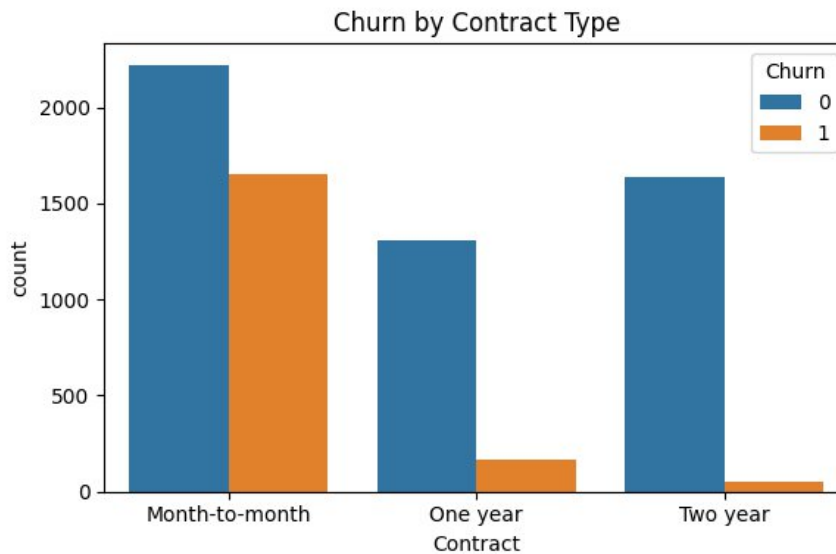
2.3 EDA Findings

- Chart 1 shows the overall split: most customers stayed but about 1 in 4 churned.
- Chart 2 shows churn by contract type. Customers on month-to-month contracts had a churn rate of around 43%, compared to about 11% for one-year contracts and only 3% for two-year contracts. This was the clearest pattern in the whole dataset.
- Chart 3 shows tenure against churn. New customers in their first year were far more likely to leave. The longer someone stays, the less likely they are to go.

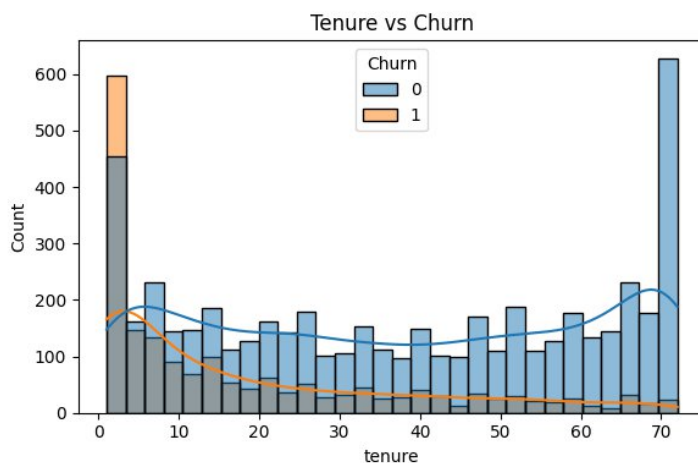
Churn Distribution



Churn by Contract



Tenure vs Churn



2.4 Feature Engineering

I created two new features to help the model. The first was `tenure_group`, which groups customers into New (0 to 12 months), Growing (13 to 24 months), Mature (25 to 48 months) and Loyal (49 months and above). The second was `total_services`, which counts how many of the 7 optional services each customer has (phone, online security, online backup, device protection, tech support, streaming TV and streaming movies). I thought customers with fewer services might be easier to lose since they are less tied in.

2.5 Modelling Approach

- Encoded all text columns using LabelEncoder and removed `customerID`
- Split the data 80/20 with `stratify=y` to keep the churn ratio equal in both sets (5,625 training, 1,407 testing)

- Trained GBM with default settings as a baseline
- Trained XGBoost with learning_rate=0.05, max_depth=4, n_estimators=500, scale_pos_weight=3 and early_stopping_rounds=20

3. Results

3.1 Model Comparison

Metric	GBM	XGBoost (Tuned)	Winner
Accuracy	0.796	0.717	GBM
Precision	0.641	0.481	GBM
Recall	0.529	0.818	XGBoost
F1 Score	0.580	0.606	XGBoost
ROC-AUC	0.839	0.838	Tie

3.2 Why I Chose XGBoost

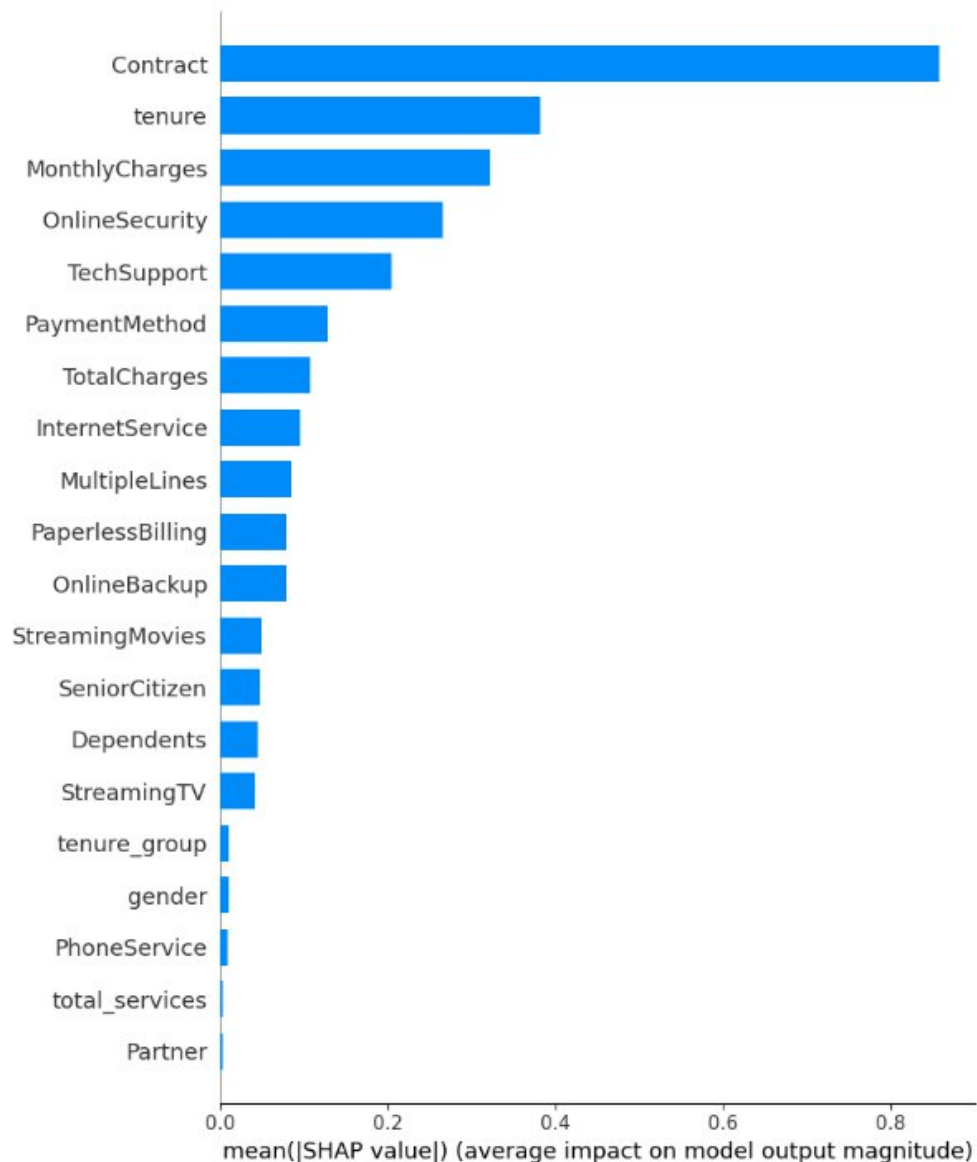
At first glance GBM looks better because it has higher Accuracy (0.796 vs 0.717) and Precision (0.641 vs 0.481). But the metric that matters most here is Recall, and XGBoost won clearly with 0.818 vs 0.529.

Recall tells you how many of the actual churners the model managed to catch. GBM only caught about 53 out of every 100 real churners, meaning it missed almost half of them. XGBoost caught about 82 out of every 100. In a real business setting, every cherner the model misses is a customer who leaves without the company ever trying to keep them. That is the more costly mistake. XGBoost flags more loyal customers as risks too (lower Precision), but the worst that happens is they receive an unnecessary offer, which is a much smaller cost.

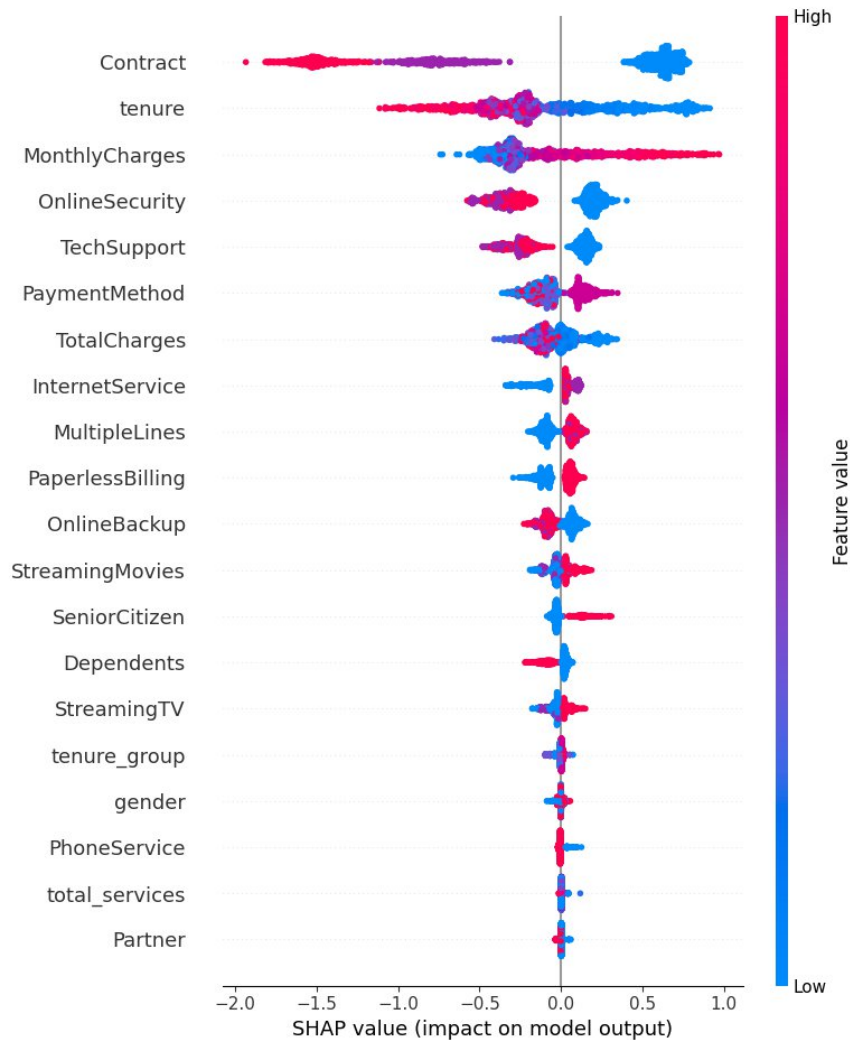
4. SHAP Explainability

After picking XGBoost as the final model, I used SHAP to understand why it was making the predictions it did. SHAP assigns a score to each feature for every single prediction, showing whether that feature pushed the prediction towards churn or away from it.

SHAP bar chart



SHAP Beeswarm plot



What the SHAP plots showed

- Contract type came out as the most important feature by far. Being on a month-to-month contract pushed churn probability up significantly. These customers have nothing keeping them locked in.
- Tenure was the second biggest factor but in the opposite direction. The longer someone has been a customer, the lower their churn probability. Long-term customers have built up habits and bundled services that make switching more effort than it is worth.
- Monthly charges also had a clear effect. Higher charges pushed churn probability up. Customers paying more seem to question whether they are getting value for money.
- Total services worked against churn. The more services a customer had, the less likely they were to leave, which makes sense since switching becomes more disruptive when you rely on multiple services.
- Having online security or tech support individually also reduced churn risk, suggesting these services make customers feel more looked after.

5. Business Recommendations

Recommendation 1: Offer Contract Upgrades to Month-to-Month Customers

Since month-to-month customers churn at around 43%, this is the group the business should focus on first. I would suggest identifying customers who are on month-to-month contracts, have been with the company for less than a year and have higher monthly bills, then reaching out with an incentive to move to a longer contract. This could be something like a discounted rate for the first few months on an annual plan or a free service add-on. Even getting a small percentage of these customers to switch would reduce churn noticeably.

Recommendation 2: Improve How New Customers Are Onboarded

The data clearly shows that the first 12 months are when customers are most likely to leave. The company should put more effort into this window. This could mean sending helpful tips about the services they signed up for, having someone check in after the first month, and making support easy to reach early on. Getting customers to actually use and value their services in the first year makes them much more likely to stay long term.

Recommendation 3: Review Pricing for High-Bill Month-to-Month Customers

High monthly charges combined with a flexible contract is a risky combination based on the SHAP findings. The business should flag customers who are in the top quarter of monthly spend and still on month-to-month contracts, then proactively offer them a pricing review or a bundled package that gives them better value. Reaching out before they decide to leave is much more effective than trying to win them back after.

6. Conclusion

Overall, I think this project showed how useful gradient boosting models can be for a real business problem like churn prediction. XGBoost with proper class imbalance handling, ended up being a much better fit for this task than a standard GBM, mainly because it caught far more actual churners (Recall of 0.818 vs 0.529). The SHAP analysis also made the results feel trustworthy since the top features, contract type, tenure and monthly charges, all make logical sense from a business perspective.

If I had more time, I would explore SMOTE as an alternative to `scale_pos_weight`, try LightGBM and CatBoost for comparison and look into building a simple API so the model could be used in a live system to score customers automatically.

7. References

- Chen, T. and Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. KDD 2016.
- Lundberg, S. and Lee, S. (2017). A Unified Approach to Interpreting Model Predictions. NeurIPS 2017.
- Friedman, J. (2001). Greedy function approximation: A gradient boosting machine. Annals of Statistics.
- Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. JMLR 12.
- IBM Telco Customer Churn Dataset. Kaggle. <https://www.kaggle.com/datasets/blastchar/telco-customer-churn>
- XGBoost Documentation. <https://xgboost.readthedocs.io>